TECNOLÓGICO NACIONAL DE MÉXICO
INSTITUTO TECNOLÓGICO DE TIJUANA

SUBDIRECCIÓN ACADÉMICA
DEPARTAMENTO DE SISTEMAS Y COMPUTACIÓN

SEMESTRE:
Enero - Junio 2026

CARRERA:
Ingeniería en Sistemas Computacionales

MATERIA:
Patrones de diseño de software

TÍTULO DE LA ACTIVIDAD:
Actividad 2 Singleton

UNIDAD A EVALUAR:
2

NOMBRE Y NÚMERO DE CONTROL DEL ALUMNO

Roberto Chavez Moreno 22210292

NOMBRE DEL MAESTRO (A):

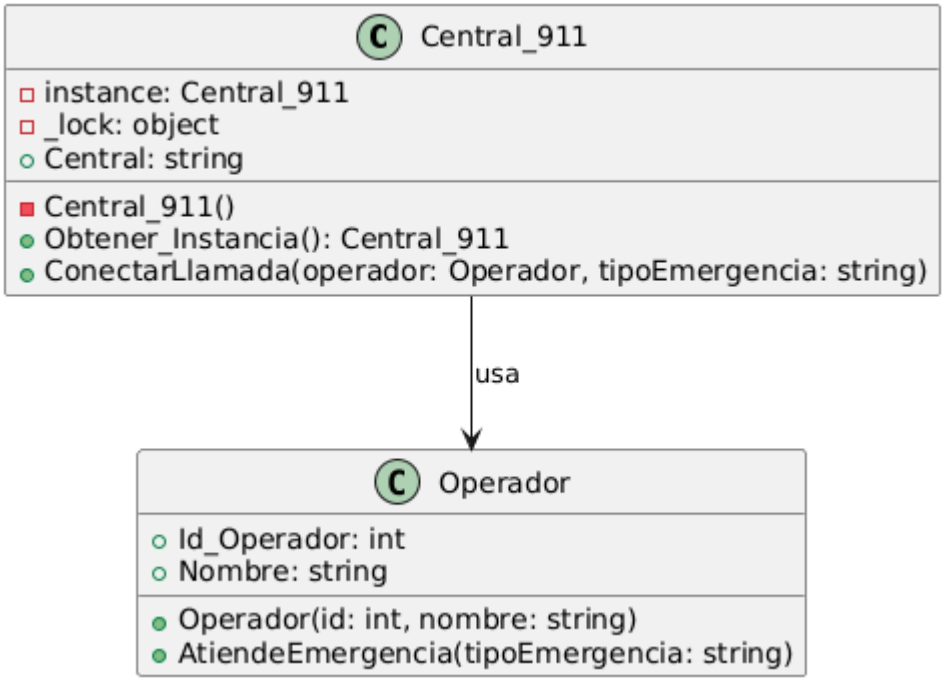
Maribel Guerrero Luis

Introducción

El patrón de diseño Singleton es un patrón creacional cuyo objetivo es garantizar que una clase tenga una sola instancia en todo el programa y proporcionar un punto global de acceso a ella.

En este proyecto se implementa una simulación de una central de emergencias 911, donde solo puede existir una única central que reciba las llamadas y las asigne a operadores disponibles.

Diagrama UML



1.El atributo estático de la instancia

```
private static Central_911 instance;
```

Este atributo es estático porque pertenece a la clase, no a los objetos.

Su función es almacenar la única instancia de **Central_911** que existirá durante toda la ejecución del programa.

Al ser estático, todas las llamadas al método **Obtener_Instancia()** comparten la misma variable, garantizando que solo se cree una instancia.

2.El constructor

```
private Central_911()
{
    Central = "Central 911";
}
```

El constructor es privado para evitar que desde fuera de la clase se pueda usar new **Central_911()**.

Así, la única forma de obtener una instancia es a través del método estático **Obtener_Instancia()**, que controla la creación única.

3.¿Por qué no se pueden crear múltiples objetos?

No se pueden crear múltiples objetos porque:

- **Constructor privado** → Impide usar new desde fuera.
- **Instancia estática privada** → Solo existe una variable que guarda el objeto.
- **Método `Obtener_Instancia()` con `lock`** → Garantiza que aunque varios hilos intenten crear la instancia al mismo tiempo, solo se creará una.
- **Double-check locking** → Primero verifica si la instancia es `null`, luego usa `lock` y verifica otra vez antes de crear.

```
C:\Users\Rober\source\repos\

Enviando bomberos.

Llamada conectada con el operador Laura
Operador Laura atendiendo emergencia de tipo: Accidente
Enviado paramedicos y oficiales.

Llamada conectada con el operador Carlos
Operador Carlos atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador María
Operador María atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador José
Operador José atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Ana
Operador Ana atendiendo emergencia de tipo: Accidente
Enviado paramedicos y oficiales.

Llamada conectada con el operador Pedro
Operador Pedro atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

Llamada conectada con el operador Sofía
Operador Sofía atendiendo emergencia de tipo: Incendio
Enviando bomberos.

Llamada conectada con el operador Miguel
Operador Miguel atendiendo emergencia de tipo: Violeta
Enviando una patrulla.

Llamada conectada con el operador Pedro
Operador Pedro atendiendo emergencia de tipo: Accidente
Enviado paramedicos y oficiales.

Llamada conectada con el operador Sofía
Operador Sofía atendiendo emergencia de tipo: Intento de suicidio
Enviando unidades de apoyo y rescate

================================
VERIFICACION DE SINGLETON:
================================
ReferenceEquals: True
HashCode Llamada1: 46104728
HashCode Llamada2: 46104728
Total operadores: 8
Total llamadas: 15
================================
```

Program.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Singleton_911
{
    internal class Program
    {
        static void Main(string[] args)
```

```
        {
            Central_911 Llamada1 =
Central_911.Obtener_Instancia();
            Central_911 Llamada2 =
Central_911.Obtener_Instancia();

            Operador op1 = new Operador(1, "Laura");
            Operador op2 = new Operador(2, "Carlos");
            Operador op3 = new Operador(3, "María");
            Operador op4 = new Operador(4, "José");
            Operador op5 = new Operador(5, "Ana");
            Operador op6 = new Operador(6, "Pedro");
            Operador op7 = new Operador(7, "Sofía");
            Operador op8 = new Operador(8, "Miguel");

            Console.WriteLine("=== LLAMADAS A LA CENTRAL 911
===\n");

            Llamada1.ConectarLlamada(op1, "Incendio");
            Llamada2.ConectarLlamada(op2, "Violeta");
            Llamada1.ConectarLlamada(op3, "Accidente");
            Llamada2.ConectarLlamada(op4, "Intento de suicidio");
            Llamada1.ConectarLlamada(op5, "Incendio");
            Llamada2.ConectarLlamada(op1, "Accidente");
            Llamada1.ConectarLlamada(op2, "Violeta");
            Llamada2.ConectarLlamada(op3, "Incendio");
            Llamada1.ConectarLlamada(op4, "Violeta");
            Llamada2.ConectarLlamada(op5, "Accidente");
            Llamada1.ConectarLlamada(op6, "Intento de suicidio");
            Llamada2.ConectarLlamada(op7, "Incendio");
            Llamada1.ConectarLlamada(op8, "Violeta");
            Llamada2.ConectarLlamada(op6, "Accidente");
            Llamada1.ConectarLlamada(op7, "Intento de suicidio");

            // VERIFICACIÓN

Console.WriteLine("\n================================");
            Console.WriteLine("VERIFICACIÓN DE SINGLETON:");

Console.WriteLine("================================");
            Console.WriteLine("ReferenceEquals: " +
ReferenceEquals(Llamada1, Llamada2));
```

```
            Console.WriteLine($"HashCode Llamada1:
{Llamada1.GetHashCode()}");
            Console.WriteLine($"HashCode Llamada2:
{Llamada2.GetHashCode()}");
            Console.WriteLine($"Total operadores: 8");
            Console.WriteLine($"Total llamadas: 15");

Console.WriteLine("================================");

            Console.ReadKey();
        }
    }
}
```

Central_911.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Singleton_911
{
    public class Central_911
    {
        private static Central_911 instance;
        private static readonly object _lock = new object();

        public string Central { get; private set; }

        private Central_911()
        {
            Central = "Central 911";
        }

        public static Central_911 Obtener_Instancia()
        {
            if (instance == null)
            {
                lock (_lock)
                {
```

```
                    if (instance == null)
                    {
                        instance = new Central_911();
                    }
                }
            }
            return instance;
        }

        public void ConectarLlamada(Operador operador, string
tipoEmergencia)
        {
            Console.WriteLine("\nLlamada conectada con el operador
" + operador.Nombre);
            operador.AtiendeEmergencia(tipoEmergencia);
        }
    }
}
```

Operador.cs

```
using System;
using System.Collections.Generic;
using System.Linq;
using System.Text;
using System.Threading.Tasks;

namespace Singleton_911
{
    public class Operador
    {
        public int Id_Operador { get; set; }
        public string Nombre { get; set; }

        public Operador(int id, string nombre)
        {
            Id_Operador = id;
            Nombre = nombre;
        }

        public void AtiendeEmergencia(string tipoEmergencia)
        {
```

```
            Console.WriteLine($"Operador {Nombre} atendiendo
emergencia de tipo: {tipoEmergencia}");
            switch (tipoEmergencia)
            {
                case "Intento de suicidio":
                    Console.WriteLine("Enviando unidades de apoyo
y rescate");
                    break;
                case "Incendio":
                    Console.WriteLine("Enviando bomberos.");
                    break;
                case "Accidente":
                    Console.WriteLine("Enviado paramedicos y
oficiales.");
                    break;
                case "Violeta":
                    Console.WriteLine("Enviando una patrulla.");
                    break;
                default:
                    Console.WriteLine("Tipo de emergencia no
reconocido.");
                    break;
            }
        }
    }
}
```

Conclusiones

En esta actividad comprendi cómo funciona el patrón Singleton, que garantiza que solo exista una única instancia de una clase. Aprendí que el constructor privado evita crear objetos con new, y que la instancia estática guarda el único objeto. Para cumplir con la actividad, agregué más operadores y más llamadas. Al verificar con ReferenceEquals, comprobé que Llamada1 y Llamada2 son el mismo objeto, confirmando que solo hay una central 911.